

SOLID Principles & Generics

F. Hamid

SOLID

- Single Responsibility Principle
 - Open-Closed Principle
 - Liskov's Substitute Principle
 - Interface Segregation Principle
 - Dependency Inversion Principle
-
- SOLID isn't a set of Java syntax rules.
 - Your program will compile even if you completely ignore SOLID.
 - These are design principles that help us make better decisions as our software becomes larger.

Single Responsibility Principle

- simple starting definition:
 - **A class should have one primary responsibility.**
- slightly more formal version:
 - **A class should have one reason to change.**

Single Responsibility Principle

- In the initial version of the **Student** class, we bundled several responsibilities into the class that were not naturally or tightly associated with representing a Student.
- So, we **refactored** the class and moved some of those responsibilities into separate classes.

Current Structure of Our Design

- Student Class
 - represents/stores student information
- EmailGenerator Class
 - handles email-generation logic
- SaveDataToFile Class
 - handles file-writing logic

Single Responsibility does not mean single method.

It means the methods and data of a class should generally work together toward one cohesive responsibility.

Open-Closed Principle (SRP)

The [Open–Closed Principle \(OCP\)](#) states that software entities like classes, modules, and functions should be open for extension but closed for modification.

What kind of object can we save using this method?

```
public class SaveDataToFile{  
    public static void saveDataToFile(String path, String filename, Student sdata) {  
        String fileName = path + "/" + filename + ".txt";  
>    try (PrintWriter writer = new PrintWriter(fileName)) {  
    } catch (FileNotFoundException e) {  
        System.out.println(x: "Could not create the file.");  
    }  
}  
}
```

Student and saveDataToFile are still **tightly coupled**.

What happens when Faculty arrives?

=> Can we **reuse** the **saveDataToFile** method to store Faculty information?

OPEN-CLOSED Principle => [after the new refactoring/adjustment]

Our file-saving mechanism is **now more open to extension**.

We can introduce another Savable type (say Staff) without modifying the existing saveDataToFile() method.

OCP => Core Concepts

- **Open for extension:** You can add new features or behaviors to the system.
- **Closed for modification:** You do this without changing existing, tested source code.
- **Goal:** Avoid breaking old features when adding new ones.
- **How It Works**
 - **Abstractions:** Use [interfaces](#) or abstract base classes.
 - **Polymorphism:** Write new classes that implement the interface instead of editing old logic.
 - **Composition:** Combine objects to build new behaviors safely.
- **Benefits**
 - **Lower risk:** Existing code stays untouched, reducing new bugs.
 - **Easy maintenance:** Adding features requires writing new files rather than refactoring legacy blocks.

Switching Gear

Generics => A Java Language Feature

SOLID is about software-design principles.

Generics

- You have already seen them.
 - `ArrayList<String> names = new ArrayList<>();`
 - `ArrayList<Integer> numbers = new ArrayList<>();`
 - `ArrayList<Student> students = new ArrayList<>();`
 - `ArrayList<Faculty> faculty = new ArrayList<>();`
- ArrayList is a generic class. We can create an ArrayList for many different reference types by specifying the element type.

Generics >> DRY Principle

Don't Repeat Yourself

“When the same concept or logic is being repeated unnecessarily, check whether it can be represented once and reused.”

Our First Generic Class

```
public class ApiResponse<T> {  
  
    private String status;  
    private String message;  
    private T data;  
  
    public ApiResponse(String status, String message, T data) {  
        this.status = status;  
        this.message = message;  
        this.data = data;  
    }  
  
    public T getData() {  
        return data;  
    }  
}
```

T is a type parameter. Think of it as a placeholder for a type that we will specify later.

- `ApiResponse<Student> s1ApiResponse = new
ApiResponse<Student>("active", "good academic standing", s1);`
- `System.out.println(s1ApiResponse.getMessage());`
- `ApiResponse<Faculty> f1ApiResponse = new
ApiResponse<Faculty>("teaching", "eligible for promotion", f1);`
- `System.out.println(f1ApiResponse.getData() + " \n ==> " +
f1ApiResponse.getMessage());`

Polymorphism & Generics

- Don't confuse these two mechanisms.
- Savable uses an interface and polymorphism.
- ApiResponse<T> uses Java Generics.
- Both help us write flexible code, but they solve different kinds of problems.

Generics

- Reading::
- <https://medium.com/@dheeraj.bartwal91/java-generics-explained-from-basics-to-advanced-with-real-world-examples-bff9dd5c8c51>
- <https://docs.oracle.com/javase/tutorial/java/generics/>